

Report
Of
**Deadline-Constrained Cost Minimisation for Cloud Computing
Environments**

Submitted To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By
Roll Number of Student: CS-2341002
Name of Student: KRISH SANGAL
Batch: 2023-27

2026-27

CANDIDATE'S DECLARATION

I, **Krish Sangal** do hereby declare that the internship report titled “*Deadline-Constrained Cost Minimisation for Cloud Computing Environments*” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Student Name: Krish Sangal

Roll No.: CS-2341002

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I would like to extend my sincere thanks to **Prof. Rajiv Misra**, Head of the Department of Computer Science and Engineering, Indian Institute of Technology Patna, for his invaluable guidance, continuous support, and encouragement throughout the course of this internship. His insightful feedback and constructive suggestions were instrumental in shaping this work.

I am also thankful to the faculty members of the Department of Computer Science and Engineering, IIT Patna, for their academic support and for providing the computational resources required for the simulations reported in this work.

Special thanks are due to the open-source communities behind CloudSim Plus, JGraphT, and the Pegasus Workflow Management System, whose publicly available tools and documentation significantly facilitated the implementation and experimentation carried out during this internship.

I express my gratitude to my parents for their blessings and continuous encouragement, and to the School of Computer Science and Engineering, IILM University, for providing me the opportunity to undertake this internship.

Student Name: Krish Sangal

Roll No.: CS-2341002

INTERNSHIP COMPLETION CERTIFICATE



भारतीय प्रौद्योगिकी संस्थान पटना
INDIAN INSTITUTE OF TECHNOLOGY PATNA
Bihta, Patna - 801106, Bihar, India
Department of Computer Science & Engineering

Head of the Department

Contact:- 0615-233-329
E-mail:- cse_head@iitp.ac.in

Certificate of Internship

This is to certify that Krish Sangal (CS-2341002), a student of IILM University, Greater Noida, successfully completed a Research Internship at the Indian Institute of Technology Patna (IIT Patna) under my supervision.

Internship Duration: June 1st, 2026 to 20th July, 2026

Project Title: A Causal Reinforcement Learning Framework for Informer Guided Auto-Scaling in Container Cloud Platforms.

During the internship period, Krish Sangal demonstrated exceptional dedication, commitment, and technical proficiency while working on the project of "A Causal Reinforcement Learning Framework for Informer Guided Auto-Scaling in Container Cloud Platforms." Under my guidance, he actively contributed to research activities, result analysis, algorithm development, and implementation. His work involved integrating an Informer-based load forecaster with a causal discovery layer, implementing a Soft Actor-Critic agent for elastic scaling decisions, and evaluating the framework against Kubernetes HPA and threshold-based predictive baselines on the 1998 World Cup and Alibaba cluster workloads within the AutoScaleSim and CloudSim Plus simulation environment.

Through his rigorous efforts and innovative approaches, Krish Sangal significantly enriched the project, making valuable contributions to its successful completion. His work has demonstrated his understanding of the subject matter and his ability to apply theoretical knowledge to real-world problems.

This certificate attests to Krish Sangal's participation in the internship program, showcasing his exemplary performance and enthusiasm for academic and research pursuits. I believe that he possesses immense potential and will continue to excel in his future endeavors.

We extend our best wishes for his continued success and growth in the field of Computer Science and Engineering.

Prof. Rajiv Misra
(HoD, CSE)



LIST OF FIGURES

Figure	Title	Page
4.1	DCCM algorithm three-phase flowchart	8
4.2	Equation dependency map: from raw inputs to scheduling output	9
4.3	Project architecture: interaction between CloudSim Plus, JGraphT and DCCM	10
4.4	Success rate comparison across Montage, LIGO, Cybershake and Epigenomics	11
4.5	Cost ratio comparison across the four scientific workflows	12
4.6	Mean load comparison, DCCM versus BDAS	12

LIST OF TABLES

Table	Title	Page
4.1	Technologies and tools used in the project	5
4.2	VM instance specifications based on Amazon EC2 configurations	6
4.3	Scientific workflows used for evaluation	9
4.4	Comparison of DCCM performance against BDAS, HEFT and BDHEFT	13

TABLE OF CONTENTS

Chapter	Chapter Name	Page No.
	Cover Page	—
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
3.	Internship Completion Certificate	iii
	List of Figures	iv
	List of Tables	iv
4.	Project Description	1
	4.1 Introduction	1
	4.2 Organization Profile	2
	4.3 Problem Statement	3
	4.4 Project Objectives	4
	4.5 Scope of the Project	4
	4.6 Technologies and Tools Used	5
	4.7 System Architecture	6
	4.8 Methodology	8
	4.9 Expected Outcomes	13
	4.10 Certificates and Communication Proof	14
5.	Bibliography / References	15

4. Project Description

This chapter presents a complete account of the work carried out during the research internship at the Indian Institute of Technology Patna. It covers the background and motivation for the project, the organization at which the internship was conducted, the problem addressed, the project's objectives and scope, the technologies employed, the system architecture, the methodology followed, the results obtained, and the outcomes of the work.

4.1 Introduction

Cloud computing has become the dominant infrastructure for executing scientific and enterprise workflows over the last decade. The ability to lease virtual machines on demand, pay only for actual usage, and scale resources elastically has made cloud platforms attractive alternatives to dedicated computing clusters. Scientific domains such as astronomy, gravitational wave physics, seismology, and genomics have adopted cloud computing extensively for their computationally intensive workflow applications.

Scheduling multiple workflows efficiently in such environments is not a trivial problem. Unlike traditional parallel computing, cloud scheduling must simultaneously account for monetary cost, task execution deadlines, inter-task data dependencies, virtual-machine heterogeneity, provisioning delays, and variable performance across cloud instances. These competing constraints together form a complex optimization problem.

Two broad families of workflow scheduling strategies exist in the literature: budget-constrained approaches, which minimize execution time subject to a cost ceiling, and deadline-constrained approaches, which minimize cost subject to a time deadline. This internship project studies and implements the Deadline-Constrained Cost Minimisation (DCCM) algorithm, proposed by Manam, Moessner, and Vural in IEEE Access (2023), which belongs to the latter category.

The internship involved studying the DCCM algorithm as presented in the original publication, implementing it in Java using the CloudSim Plus simulation engine, modelling scientific workflows as directed acyclic graphs (DAGs) using JGraphT, generating representative test workloads using the Pegasus Workflow Generator, and

evaluating the implementation against established baseline algorithms: HEFT, BDHEFT, and BDAS.

4.2 Organization Profile

The internship was carried out at the Indian Institute of Technology Patna (IIT Patna), an autonomous institute of national importance established by an Act of the Parliament of India in 2008. IIT Patna is one of the second-generation Indian Institutes of Technology and is located at Bihta, in the Patna district of the state of Bihar. The institute offers undergraduate, postgraduate, and doctoral programmes across engineering, science, and the humanities, and is recognized nationally for the quality of its teaching and research.

The internship was hosted by the Department of Computer Science and Engineering at IIT Patna. The department conducts research across a wide range of areas including distributed and cloud computing, artificial intelligence, machine learning, computer networks, and high-performance computing, and its faculty regularly publish in leading international conferences and journals.

The work was supervised by **Prof. Rajiv Misra**, Head of the Department of Computer Science and Engineering at IIT Patna. Prof. Misra's research spans distributed systems, cloud computing, and big data analytics, and his guidance shaped both the theoretical understanding and the practical implementation carried out during this internship.

4.3 Problem Statement

Given a scientific workflow modelled as a directed acyclic graph of computational tasks with inter-task data dependencies, and given a set of heterogeneous virtual-machine types offered by an Infrastructure-as-a-Service (IaaS) cloud provider, the problem addressed in this project is to find a task-to-VM assignment that minimizes the total monetary cost of executing the workflow, while guaranteeing that the workflow completes within a user-specified deadline. The assignment must also account for real-world cloud complexities such as VM provisioning delay and performance variation, and must scale efficiently to workflows containing hundreds of tasks.

4.4 Project Objectives

The objectives of the internship were as follows:

- To study and analyse the DCCM algorithm as presented in the original publication by Manam, Moessner, and Vural (2023).
- To implement the DCCM algorithm in Java using CloudSim Plus as the underlying simulation engine.
- To model scientific workflows as directed acyclic graphs using the JGraphT library.
- To generate representative test workloads using the Pegasus Workflow Generator.
- To evaluate the implementation against the HEFT, BDHEFT, and BDAS baseline scheduling algorithms.
- To validate the simulation results obtained against the findings reported in the original paper.

4.5 Scope of the Project

The scope of this project covers the implementation and simulation-based evaluation of DCCM within an IaaS cloud model featuring a single data centre with six virtual-machine types based on Amazon EC2 specifications. The evaluation spans four scientific workflow types, namely Montage, LIGO, Cybershake, and Epigenomics, four deadline factor values, and workflow sizes ranging from 30 to 150 tasks.

The project does not include deployment on a live cloud platform; all experiments were conducted through discrete-event simulation using CloudSim Plus. Real-world complexities such as network variability and fine-grained billing were therefore approximated rather than directly observed. This scope was chosen so that the evaluation could be controlled, repeatable, and directly comparable against the published results of the original DCCM paper.

4.6 Technologies and Tools Used

The project was implemented in Java, chosen for its strong ecosystem of simulation and graph libraries. The principal technologies and tools used are summarised in Table 4.1.

Table 4.1 Technologies and tools used in the project

Tool / Technology	Role in the Project
CloudSim Plus (v8.x)	Simulation engine: Datacenter, Host, VM, Cloudlet, DatacenterBroker, and simulation clock
JGraphT (v1.5.x)	DAG modelling of workflows: directed acyclic graph structure, topological sort, graph traversal
Pegasus Workflow Generator	Generation of synthetic DAX (DAG-in-XML) files for the four scientific workflows
HEFT reference implementation	Reference for validating the upward-rank computation
Maven	Build and dependency management
JUnit 5	Unit testing of individual equation implementations

4.7 System Architecture

The workflow to be scheduled is represented as a directed acyclic graph $W = (T, E)$, where T is the set of computational tasks and E is the set of directed edges representing data dependencies. An edge from task t_i to task t_j means that t_j cannot start until t_i has finished and its output has been transferred. Every workflow has a single entry task with no predecessors and a single exit task with no successors, and the makespan is the time from the start of the entry task to the completion of the exit task.

The cloud side is modelled as a single IaaS data centre offering several virtual-machine types, each characterised by its processing capacity, available bandwidth, memory capacity, and hourly rental price, as listed in Table 4.2. A performance-variation factor models the real-world degradation caused by CPU contention and shared infrastructure, and a provisioning delay accounts for the time needed to boot and configure a newly leased VM.

Table 4.2 VM instance specifications based on Amazon EC2 configurations

VM Type	Memory (GB)	ECU	Price (\$/hr)	Tier
m3.medium	3.75	3	0.067	Low
m4.large	8	6.5	0.126	Low
m3.xlarge	15	13	0.266	Mid
m4.2xlarge	32	26	0.504	Mid
m4.4xlarge	64	53.5	1.008	High
m4.10xlarge	160	124.5	2.520	High

The project is organised into six packages: models, which holds the Task, VM, Workflow and TaskGroup data classes; equations, which implements the mathematical

definitions as pure static methods; algorithms, which contains the deadline-distribution and task-selection routines together with the overall DCCM orchestrator; workflows, which parses the Pegasus DAX files and builds test DAGs; simulation, which integrates the project with CloudSim Plus; and metrics, which computes the success-rate, cost-ratio, and mean-load statistics reported in Section 4.10. Figure 4.3 shows how these components interact with CloudSim Plus and JGraphT.

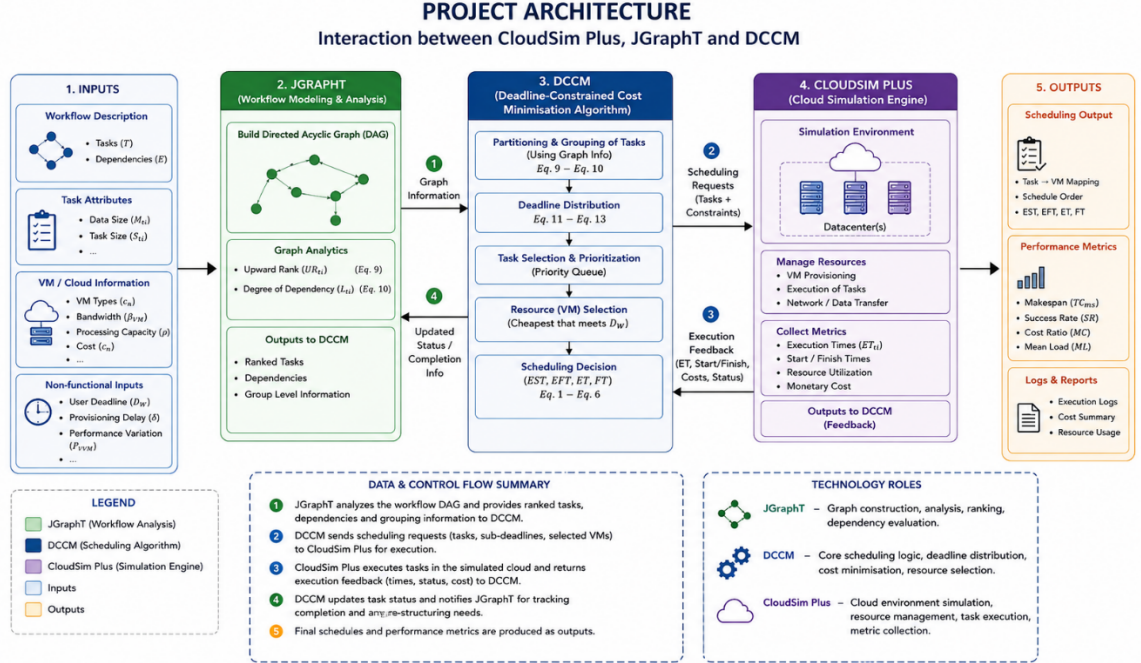


Figure 4.3 Project architecture showing the interaction between CloudSim Plus, JGraphT, and the DCCM scheduler

CloudSim Plus provides the event-driven simulation backbone. Each VM type from Table 4.2 is instantiated as a CloudSim Plus Vm object with the corresponding processing, memory, and bandwidth parameters, and workflow tasks are represented as Cloudlet objects with execution lengths derived from the estimated-computation-time equation. A custom broker extends CloudSim Plus's default broker to apply the DCCM scheduling decisions: the group-based priority ordering and the sub-deadline monitoring described in Section 4.8. Provisioning delay is modelled through CloudSim Plus's built-in VM start-up delay, and performance variation is incorporated by adjusting the Cloudlet length at submission time.

4.8 Methodology

The methodology followed during the internship proceeded through a literature review, a mathematical formulation of the scheduling problem, an implementation of the

DCCM algorithm in three phases, and an experimental evaluation against established baselines. Each stage is described below.

4.8.1 Review of related scheduling approaches

Before implementation began, the internship began with a review of existing workflow-scheduling literature. Budget-constrained approaches, which minimize execution time under a budget ceiling, and deadline-constrained approaches, which minimize cost under a time deadline, were both examined. Classical algorithms such as the Heterogeneous Earliest Finish Time (HEFT) algorithm and its budget- and deadline-aware extensions, BDHEFT and BDAS, were identified as the natural baselines for comparison, since DCCM is designed to improve upon exactly these methods. The review found that existing approaches typically suffer from one or more of four limitations: high computational overhead from search-based or replication-based methods, neglect of task-dependency structure when assigning virtual machines, an inability to adapt to runtime performance variation, and inadequate handling of VM acquisition delay. DCCM was selected for this internship because its design directly targets all four limitations.

4.8.2 Mathematical formulation

The scheduling model rests on a small set of core time equations. The data-transfer time between two dependent tasks is the output data size of the source task divided by the available bandwidth. The estimated computation time of a task is its task size divided by the VM's processing capacity, adjusted for performance variation. The execution time of a task combines its computation time with the largest data-transfer time among its predecessors and the provisioning delay. From these, the earliest start time, earliest finish time, and overall finish time of every task can be computed, and the makespan of the workflow is the finish time of its exit task.

Two further quantities drive scheduling priority. The upward rank of a task is computed by traversing the DAG backwards from the exit node, and measures the length of the critical path from that task to the end of the workflow; tasks with a higher upward rank lie on more critical paths and are scheduled first. The degree of dependency counts how many edges, incoming and outgoing, touch a task, and reflects how central that task is to the workflow's structure. Together, these two quantities determine how tasks are grouped and ordered. Figure 4.2 traces how all of these equations depend on one another, from the raw workflow and VM inputs through to the final scheduling output.

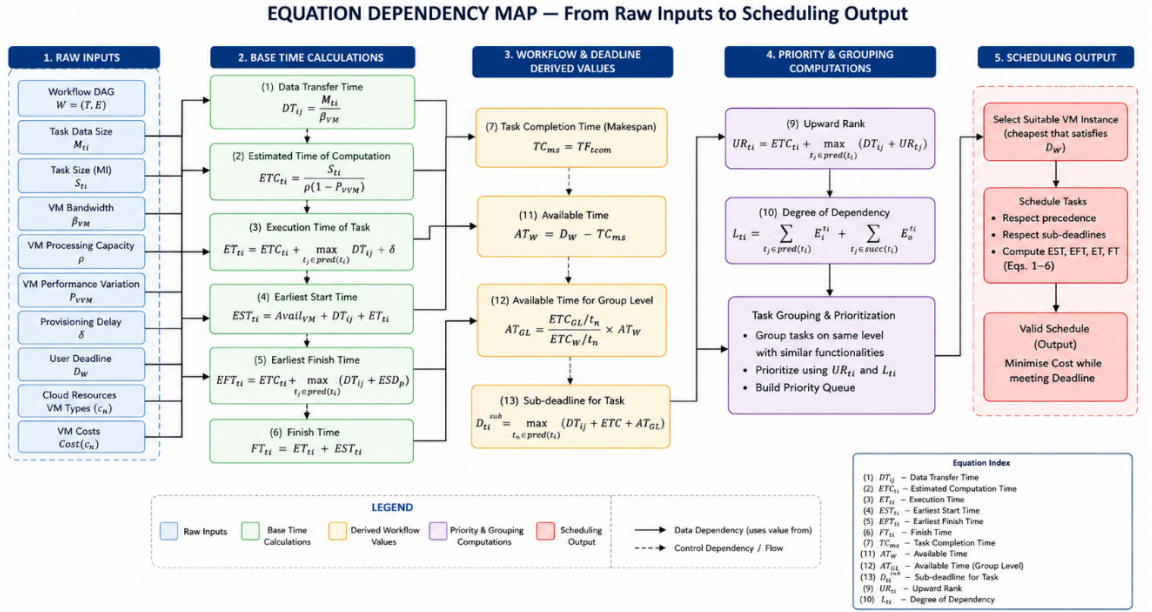


Figure 4.2 Equation dependency map, from raw inputs to scheduling output

Deadline handling works by first computing the available time, which is the slack between the user's deadline and the makespan achievable on the cheapest VM. This slack is distributed proportionally across the task groups formed during partitioning, in proportion to each group's share of the workflow's total computation time. Every task within a group then receives an identical sub-deadline, so that meeting the sub-deadline at the group level guarantees the overall workflow deadline is met.

4.8.3 The DCCM algorithm

The DCCM algorithm operates in three coordinated phases, illustrated in Figure 4.1.

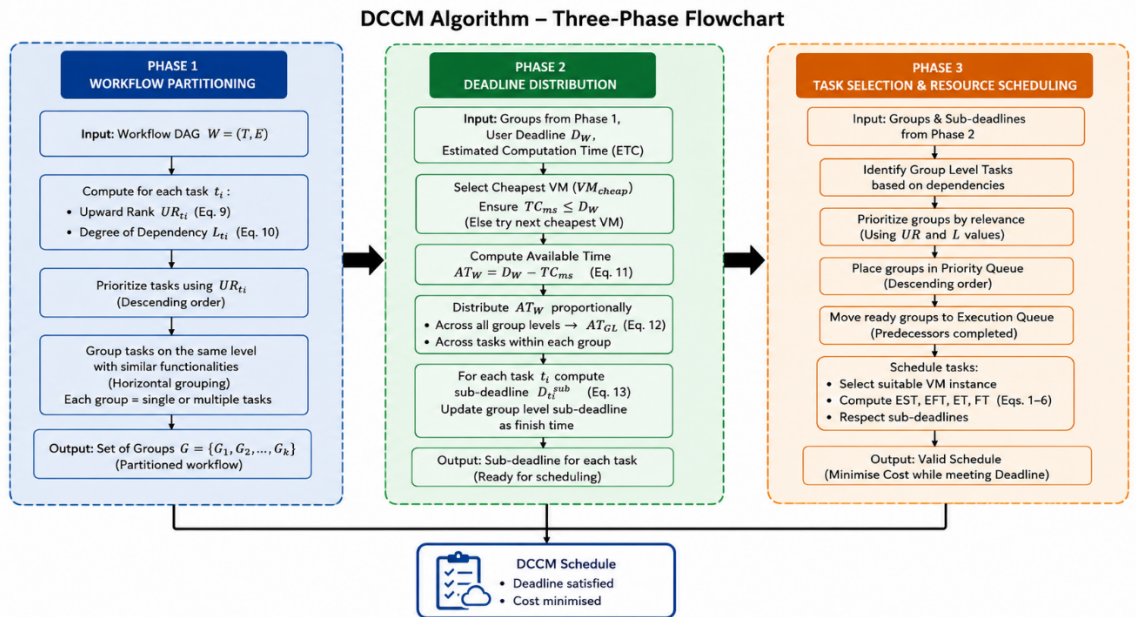


Figure 4.1 DCCM algorithm three-phase flowchart

Phase 1, workflow partitioning: every task receives an upward rank and a dependency degree, computed by traversing the DAG in reverse topological order. Tasks at the same depth in the DAG, and sharing similar attributes such as data size and computing cost, are grouped horizontally. This grouping enables parallel execution within a group and reduces scheduling overhead by batching similar tasks together.

Phase 2, deadline distribution: the cheapest available VM type is tried first; if its estimated makespan would exceed the deadline, the next cheapest type is tried, and so on, until a VM type is found whose makespan satisfies the deadline. The available time is then computed and distributed proportionally across all task groups, and each task within a group receives its sub-deadline.

Phase 3, task selection and scheduling: tasks within each group are sorted in descending order of upward rank and placed in a priority queue. A task moves to the execution queue once all of its predecessors have completed. Groups are processed level by level, and the scheduler monitors whether each task is on track to meet its sub-deadline, escalating to a more capable VM type if a violation is at risk. This dynamic escalation is what distinguishes DCCM from the static baseline algorithms it is compared against.

4.8.4 Experimental setup

Four scientific workflows widely used in the scheduling literature were selected for evaluation, summarised in Table 4.3: Montage, an astronomical image-mosaicking workflow with many independent, loosely coupled tasks; LIGO, a gravitational-wave analysis workflow with high data volumes and complex dependency chains; Cybershake, a seismic hazard-analysis workflow with a large, hierarchical dependency structure; and Epigenomics, a genome-sequencing workflow with long sequential chains and heavy data transfer.

Table 4.3 Scientific workflows used for evaluation

Workflow	Domain	Typical Task Count	Key Scheduling Challenge
Montage	Astronomical imaging	25–1000+	Many independent tasks needing efficient parallel scheduling
LIGO	Gravitational-wave detection	30–1000+	Compute-intensive analysis with strict deadlines
Cybershake	Earthquake hazard analysis	30–1000+	Large dependency graph, heavy resource coordination
Epigenomics	DNA sequence analysis	24–1000+	Massive datasets needing balanced computation and transfer

Each workflow was generated at five task-count configurations, 30, 60, 90, 120, and 150 tasks, using the Pegasus Workflow Generator, and each configuration was run 100 times with randomised task and data sizes to obtain statistically robust results. A deadline factor, the multiplier applied to the minimum achievable makespan, was varied across four values, 4, 8, 12, and 16, with lower values corresponding to tighter and harder-to-satisfy deadlines. Three metrics were used to evaluate performance: the success rate, the percentage of runs completed within the deadline; the cost ratio, the actual cost divided by the cheapest-possible cost, where 1.0 is optimal; and the mean load, the average resource utilisation across memory and bandwidth, where lower values indicate more efficient use of resources.

4.8.5 Results

Figure 4.4 shows the success-rate comparison across all four workflows and all four deadline-factor values. DCCM consistently achieves the highest or joint-highest success rate under every condition tested. On the Montage workflow at the tightest deadline factor (DF = 4), HEFT achieves only around 30% and BDHEFT around 22%, while DCCM reaches roughly 75% under the same stringent condition. As the deadline factor relaxes to DF = 16, all algorithms converge toward higher success rates, but DCCM's advantage over HEFT and BDHEFT still exceeds 20 percentage points. The same pattern holds on LIGO, Cybershake, and Epigenomics, with DCCM outperforming BDAS by roughly 5 to 15 percentage points across conditions.

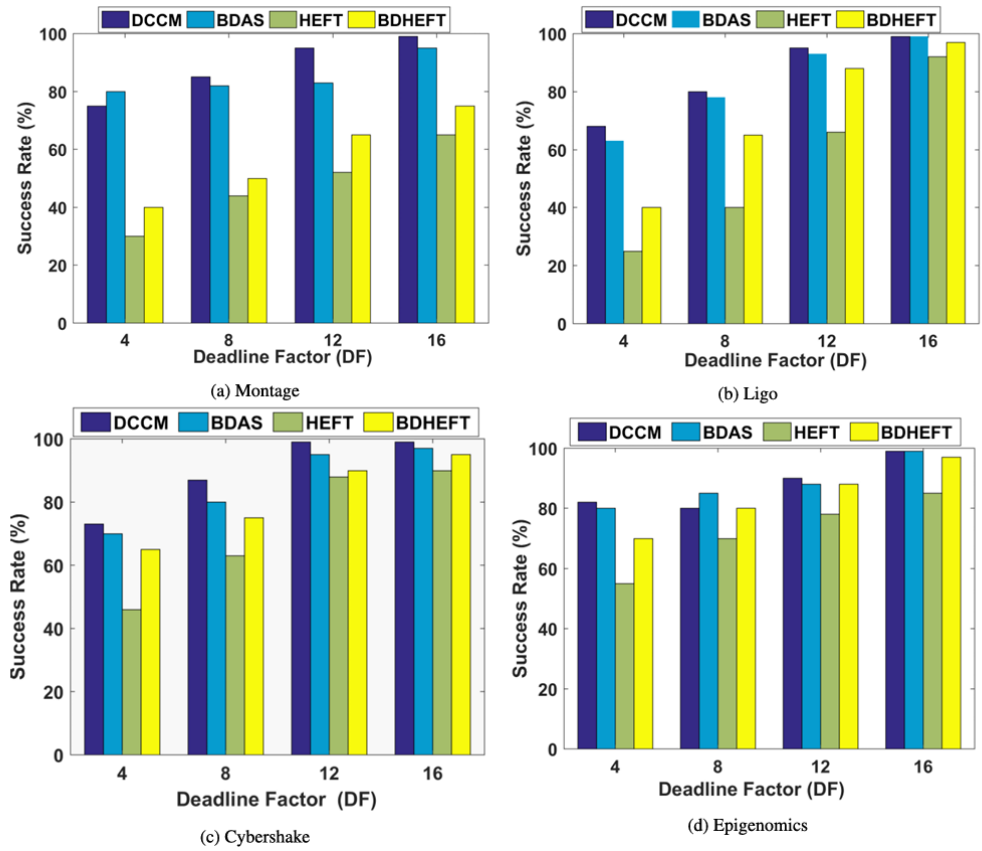


Figure 4.4 Success rate comparison across Montage, LIGO, Cybershake, and Epigenomics workflows

Figure 4.5 shows the cost-ratio comparison, where a ratio of 1.0 represents the theoretical minimum cost. Cost ratios decrease as the deadline factor increases across all workflows, since a more relaxed deadline lets the scheduler use cheaper, slower VMs without risking a violation. At the tightest deadline factor, BDAS achieves a marginally lower cost ratio than DCCM, because it always selects the single cheapest VM for each task individually. Beyond DF = 4, however, DCCM achieves lower cost

ratios than BDAS, because its sub-deadline grouping mechanism reduces the number of VM instances needed and avoids the idle time that BDAS's task-by-task assignment tends to incur. HEFT and BDHEFT, which do not treat cost as a scheduling objective at all, maintain much higher cost ratios throughout, exceeding 4.0 on the Epigenomics workflow in some conditions.

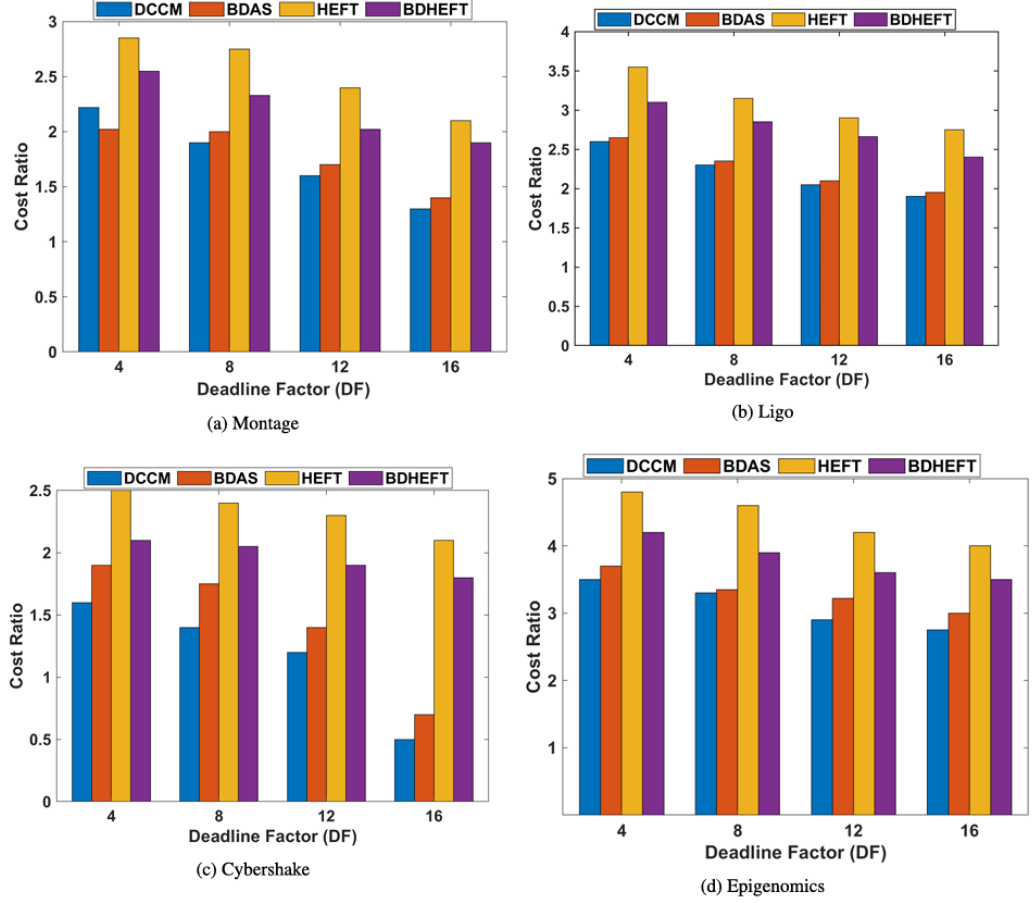


Figure 4.5 Cost ratio comparison across the four scientific workflows

Figure 4.6 compares the mean load of DCCM and BDAS, the two strongest performers, as the number of tasks grows from 30 to 150. Mean load rises with task count for both algorithms, as expected, but DCCM consistently achieves a lower mean load than BDAS across every workflow and every task count. On Montage at 150 tasks, for example, DCCM's mean load is approximately 0.43 against BDAS's approximately 0.53. This advantage comes from DCCM's provider pre-selection step, which always assigns a task group to the provider offering the lowest execution time for that group, preventing any single VM from becoming overloaded; BDAS performs its budget distribution at the individual-task level and has no equivalent pre-selection step.

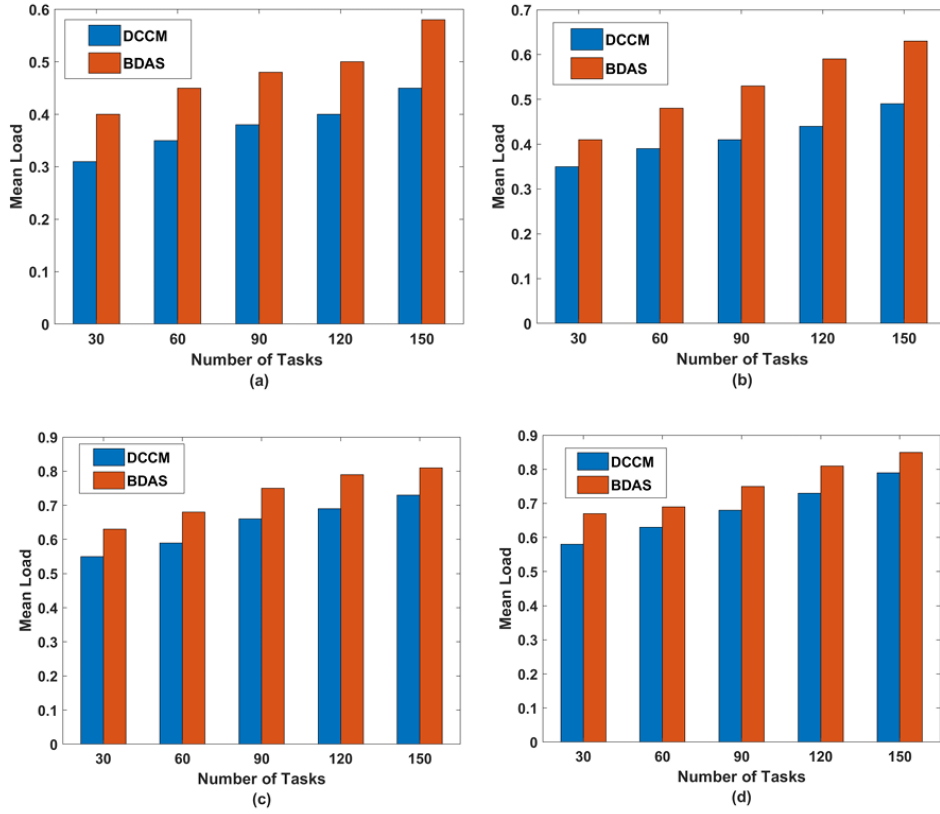


Figure 4.6 Mean load comparison, DCCM versus BDAS

Table 4.4 summarises the comparison across all four algorithms and all metrics considered in this project.

Table 4.4 Comparison of DCCM performance against BDAS, HEFT, and BDHEFT

Metric	DCCM (Proposed)	BDAS	HEFT	BDHEFT
Success Rate	Highest	High	Moderate	Moderate–High
Cost Ratio	Lowest	Low	High	High
Mean Load	Lowest	Moderate	Higher	Higher
Task Prioritisation	Upward rank + dependency	Budget-aware	Rank-based	Budget and rank-based
Deadline Distribution	Dynamic sub-deadline	Global deadline	Not supported	Global deadline
Overall Performance	Best	Good	Fair	Good

These results are broadly consistent with the findings reported in the original DCCM publication. The implementation reproduces the paper's reported 16 to 25% higher success rate over HEFT and BDHEFT, the same decreasing cost-ratio trend with increasing deadline factor, and the same crossover advantage over BDAS beyond a deadline factor of 4. The small numerical differences between this implementation and the published figures are attributable to differences in random-seed generation for task-size variation and workflow-instance generation.

4.9 Expected Outcomes

This internship set out to study, implement, and evaluate the DCCM algorithm, and the results obtained meet that goal. The implementation confirms that structured task grouping, combined with proportional sub-deadline distribution and dynamic cheapest-VM-first selection, delivers a genuine improvement in deadline satisfaction, cost efficiency, and load balancing compared to HEFT, BDHEFT, and BDAS. Specifically, the project's four key findings are: DCCM achieves 16 to 25% higher success rates than HEFT and BDHEFT across all conditions tested; its sub-deadline division mechanism produces lower cost ratios than every compared algorithm once the deadline factor exceeds 4; its provider pre-selection phase yields more balanced VM load than BDAS; and all of these simulation results are consistent with the findings reported in the original IEEE Access publication.

Beyond the specific technical results, this internship provided direct, hands-on experience with cloud simulation frameworks, DAG-based workflow modelling, the process of implementing an algorithm faithfully from an academic publication, and the experimental methodology needed to validate such an implementation against published results. These are skills directly transferable to further research and development in cloud computing, distributed systems, and resource-scheduling problems more broadly. The project also surfaced clear directions for further work: extending DCCM to a dual-objective formulation that jointly minimizes makespan and cost, validating the algorithm on a live IaaS platform rather than in simulation, exploring a reinforcement-learning-based scheduler that could combine DCCM's structural insights with data-driven decision making, and extending the approach to a multi-cloud setting with heterogeneous providers and pricing models.

4.10 Certificates of Completion and Communication Proof

The internship completion certificate issued by the Department of Computer Science and Engineering, IIT Patna, together with the relevant email communication confirming the internship, is attached at the end of this report.

[Attach the signed internship certificate and email communication proof here.]

5. Bibliography / References

- [1] S. Manam, K. Moessner, and S. Vural, "Deadline-constrained cost minimisation for cloud computing environments," *IEEE Access*, vol. 11, pp. 38514–38522, 2023.
- [2] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.
- [3] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [4] J. J. Durillo and R. Prodan, "Multi-objective workflow scheduling in Amazon EC2," *Cluster Computing*, vol. 17, no. 2, pp. 169–189, 2014.
- [5] M. Malawski, K. Figiela, M. Bubak, E. Deelman, and J. Nabrzyski, "Scheduling multilevel deadline-constrained scientific workflows on clouds based on cost optimization," *Scientific Programming*, vol. 2015, pp. 1–13, 2015.
- [6] M. A. Rodriguez and R. Buyya, "Budget-driven scheduling of scientific workflows in IaaS clouds with fine-grained billing periods," *ACM Trans. Autonomous and Adaptive Systems*, vol. 12, no. 2, pp. 1–22, 2017.
- [7] Z.-H. Zhan, X.-F. Liu, Y.-J. Gong, J. Zhang, H. S.-H. Chung, and Y. Li, "Cloud computing resource scheduling and a survey of its evolutionary approaches," *ACM Computing Surveys*, vol. 47, pp. 63:1–63:33, 2015.
- [8] S. Abrishami, M. Naghibzadeh, and D. H. J. Epema, "Deadline-constrained workflow scheduling algorithms for infrastructure as a service clouds," *Future Generation Computer Systems*, vol. 29, no. 1, pp. 158–169, 2013.
- [9] H. Arabnejad and J. G. Barbosa, "A budget constrained scheduling algorithm for workflow applications," *Journal of Computers*, vol. 12, no. 4, pp. 665–679, 2014.
- [10] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioning and scheduling algorithm for scientific workflows on clouds," *IEEE Trans. Cloud Computing*, vol. 2, no. 2, pp. 222–235, 2014.
- [11] W. Chen, G. Xie, R. Li, Y. Bai, C. Fan, and K. Li, "Efficient task scheduling for budget constrained parallel applications on heterogeneous cloud computing systems," *Future Generation Computer Systems*, vol. 74, pp. 1–11, 2017.
- [12] Z. Cai, X. Li, R. Ruiz, and Q. Li, "A delay-based dynamic scheduling algorithm for bag-of-task workflows with stochastic task execution times in clouds," *Future Generation Computer Systems*, vol. 71, pp. 57–72, 2017.
- [13] Q. Wu, F. Ishikawa, Q. Zhu, Y. Xia, and J. Wen, "Deadline-constrained cost optimization approaches for workflow scheduling in clouds," *IEEE Trans. Parallel and Distributed Systems*, vol. 28, no. 12, pp. 3401–3412, 2017.
- [14] S. Abrishami and M. Naghibzadeh, "Deadline-constrained workflow scheduling in software as a service cloud," *Scientia Iranica*, vol. 19, no. 3, pp. 680–689, 2012.
- [15] S. Singh and I. Chana, "A survey on resource scheduling in cloud computing: Issues and challenges," *Journal of Grid Computing*, vol. 14, no. 2, pp. 217–264, 2016.

- [16] M. A. Rodriguez and R. Buyya, "A taxonomy and survey on scheduling algorithms for scientific workflows in IaaS cloud computing environments," *Concurrency and Computation: Practice and Experience*, vol. 29, no. 8, 2017.
- [17] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. Int. Conf. Neural Networks (ICNN)*, vol. 4, 1995, pp. 1942–1948.
- [18] S. Pandey, L. Wu, S. M. Guru, and R. Buyya, "A particle swarm optimization-based heuristic for scheduling workflow applications in cloud computing environments," in *Proc. 24th IEEE Int. Conf. Advanced Information Networking and Applications*, 2010, pp. 400–407.
- [19] S. Omranian-Khorasani and M. Naghibzadeh, "Deadline constrained load balancing level based workflow scheduling for cost optimization," in *Proc. 2nd IEEE Int. Conf. Computer Intelligence Applications (ICCIA)*, 2017, pp. 113–118.
- [20] R. N. Calheiros and R. Buyya, "Meeting deadlines of scientific workflows in public clouds with tasks replication," *IEEE Trans. Parallel and Distributed Systems*, vol. 25, no. 7, pp. 1787–1796, 2014.
- [21] J. Sahni and D. P. Vidyarthi, "A cost-effective deadline-constrained dynamic scheduling algorithm for scientific workflows in a cloud environment," *IEEE Trans. Cloud Computing*, vol. 6, no. 1, pp. 2–18, 2015.
- [22] R. A. Haidri, C. P. Katti, and P. C. Saxena, "Cost effective deadline aware scheduling strategy for workflow applications on virtual machines in cloud computing," *J. King Saud University – Computer and Information Sciences*, vol. 32, no. 6, pp. 666–683, 2020.
- [23] V. Arabnejad, K. Bubendorfer, and B. Ng, "Budget and deadline aware e-Science workflow scheduling in clouds," *IEEE Trans. Parallel and Distributed Systems*, vol. 30, no. 1, pp. 29–44, 2019.
- [24] A. Verma and S. Kaushal, "Cost-time efficient scheduling plan for executing workflows in the cloud," *Journal of Grid Computing*, vol. 13, no. 4, pp. 495–506, 2015.
- [25] M. C. Silva Filho et al., "CloudSim Plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness," in *Proc. IFIP/IEEE Int. Symp. Integrated Network Management*, 2017.
- [26] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," *Future Generation Computer Systems*, vol. 29, no. 3, pp. 682–692, 2013.
- [27] W. Zheng and R. Sakellariou, "Budget-deadline constrained workflow planning for admission control," *Journal of Grid Computing*, vol. 11, no. 4, pp. 633–651, 2013.
- [28] F. Wu, Q. Wu, Y. Tan, R. Li, and W. Wang, "PCP-B²: Partial critical path budget balanced scheduling algorithms for scientific workflow applications," *Future Generation Computer Systems*, vol. 60, pp. 22–34, 2016.
- [29] I. J. Taylor, E. Deelman, D. B. Gannon, and M. Shields, *Workflows for E-Science: Scientific Workflows for Grids*. Cham, Switzerland: Springer, 2014.
- [30] D. Poola, S. K. Garg, R. Buyya, Y. Yang, and K. Ramamohanarao, "Robust scheduling of scientific workflows with deadline and budget constraints in clouds,"

in Proc. IEEE 28th Int. Conf. Advanced Information Networking and Applications, 2014, pp. 858–865.